Retail Order Dispatch and Return Analysis

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Process retail order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

2. Objectives

• Store order details. • Validate dispatch quantities against ordered quantities. • Validate returns against dispatched quantities. • Calculate outstanding and net-shipped quantities. • Update inventory and report inconsistent records.

3. Assumptions

Each order has a unique order ID, item, ordered quantity, and unit price. Dispatches exceeding the remaining ordered quantity are rejected. Returns must not exceed accepted dispatched quantities. Inventory decreases for accepted dispatches and increases for accepted returns. The sample data deliberately includes inconsistent records for testing.

4. Algorithm

```
1. Store order records, dispatch records, return records, and stock.
2. Initialize dispatched and returned totals for each order.
3. Validate each dispatch ID and quantity.
4. Reject dispatches that would exceed the ordered quantity.
5. Validate return IDs and ensure returns do not exceed dispatches.
6. Update stock for accepted dispatches and returns.
7. Calculate outstanding = ordered - dispatched.
8. Calculate net shipped = dispatched - returned.
9. Display order status, inventory, and data-quality issues.
```

5. Flowchart (Text Representation)

```
START
  |
Read orders, dispatches, returns, and stock
  |
Validate each dispatch record
  |-- invalid/exceeds order --> Report inconsistency
  |-- valid -----------------> Update dispatched total
  |
Validate each return record
  |-- invalid/unknown/exceeds dispatch --> Report inconsistency
  |-- valid --------------------------> Update returned total
  |
Update inventory
  |
Calculate outstanding and net shipped
  |
Display summary and inconsistent records
  |
 END
```

6. Python Source Code

```
# Retail Order Dispatch and Return Analysis
# Process order quantities, dispatches, returns, outstanding quantities,
# inventory updates, and inconsistent records.

orders = {
    "ORD101": {"item": "Notebook", "ordered": 100, "unit_price": 40},
    "ORD102": {"item": "Pen", "ordered": 200, "unit_price": 10},
    "ORD103": {"item": "File", "ordered": 80, "unit_price": 25},
    "ORD104": {"item": "Marker", "ordered": 50, "unit_price": 30},
}

dispatches = [
    ("ORD101", 60),
    ("ORD101", 40),
    ("ORD102", 150),
    ("ORD103", 90),   # Inconsistent: exceeds ordered quantity
    ("ORD104", 30),
]

returns = [
    ("ORD101", 5),
    ("ORD102", 10),
    ("ORD104", 2),
    ("ORD999", 1),    # Inconsistent: unknown order ID
```

```
]

# Starting inventory is the quantity currently available before these dispatches.
inventory = {"Notebook": 200, "Pen": 300, "File": 100, "Marker": 80}

dispatch_totals = {order_id: 0 for order_id in orders}
return_totals = {order_id: 0 for order_id in orders}
issues = []
valid_dispatches = []
valid_returns = []

print("RETAIL ORDER DISPATCH AND RETURN ANALYSIS")
print("=" * 88)
print("PROCESSING DISPATCH RECORDS")
print("-" * 88)

for order_id, qty in dispatches:
    if order_id not in orders:
        issues.append(f"Dispatch {order_id}: unknown order ID.")
        print(f"INCONSISTENT: {order_id} is not present in order records.")
        continue
    if qty <= 0:
        issues.append(f"Dispatch {order_id}: invalid quantity {qty}.")
        print(f"INCONSISTENT: {order_id} has invalid dispatch quantity {qty}.")
        continue

    ordered = orders[order_id]["ordered"]
    if dispatch_totals[order_id] + qty > ordered:
        issues.append(
            f"Dispatch {order_id}: total dispatch would exceed ordered quantity "
            f"({dispatch_totals[order_id] + qty} > {ordered})."
        )
        print(f"INCONSISTENT: {order_id} dispatch rejected; total would be "
              f"{dispatch_totals[order_id] + qty}, ordered quantity is {ordered}.")
        continue

    dispatch_totals[order_id] += qty
    valid_dispatches.append((order_id, qty))
    print(f"Accepted dispatch: {order_id} -> {qty} units")

print("\nPROCESSING RETURN RECORDS")
print("-" * 88)
for order_id, qty in returns:
    if order_id not in orders:
        issues.append(f"Return {order_id}: unknown order ID.")
        print(f"INCONSISTENT: return refers to unknown order {order_id}.")
        continue
    if qty <= 0:
        issues.append(f"Return {order_id}: invalid quantity {qty}.")
        print(f"INCONSISTENT: {order_id} has invalid return quantity {qty}.")
        continue
    if return_totals[order_id] + qty > dispatch_totals[order_id]:
        issues.append(f"Return {order_id}: returned quantity exceeds dispatched quantity.")
        print(f"INCONSISTENT: {order_id} return rejected; return exceeds dispatch.")
        continue

    return_totals[order_id] += qty
    valid_returns.append((order_id, qty))
    print(f"Accepted return: {order_id} -> {qty} units")

# Inventory decreases by accepted dispatches and increases by accepted returns.
for order_id, qty in valid_dispatches:
    item = orders[order_id]["item"]
    inventory[item] -= qty
for order_id, qty in valid_returns:
    item = orders[order_id]["item"]
    inventory[item] += qty

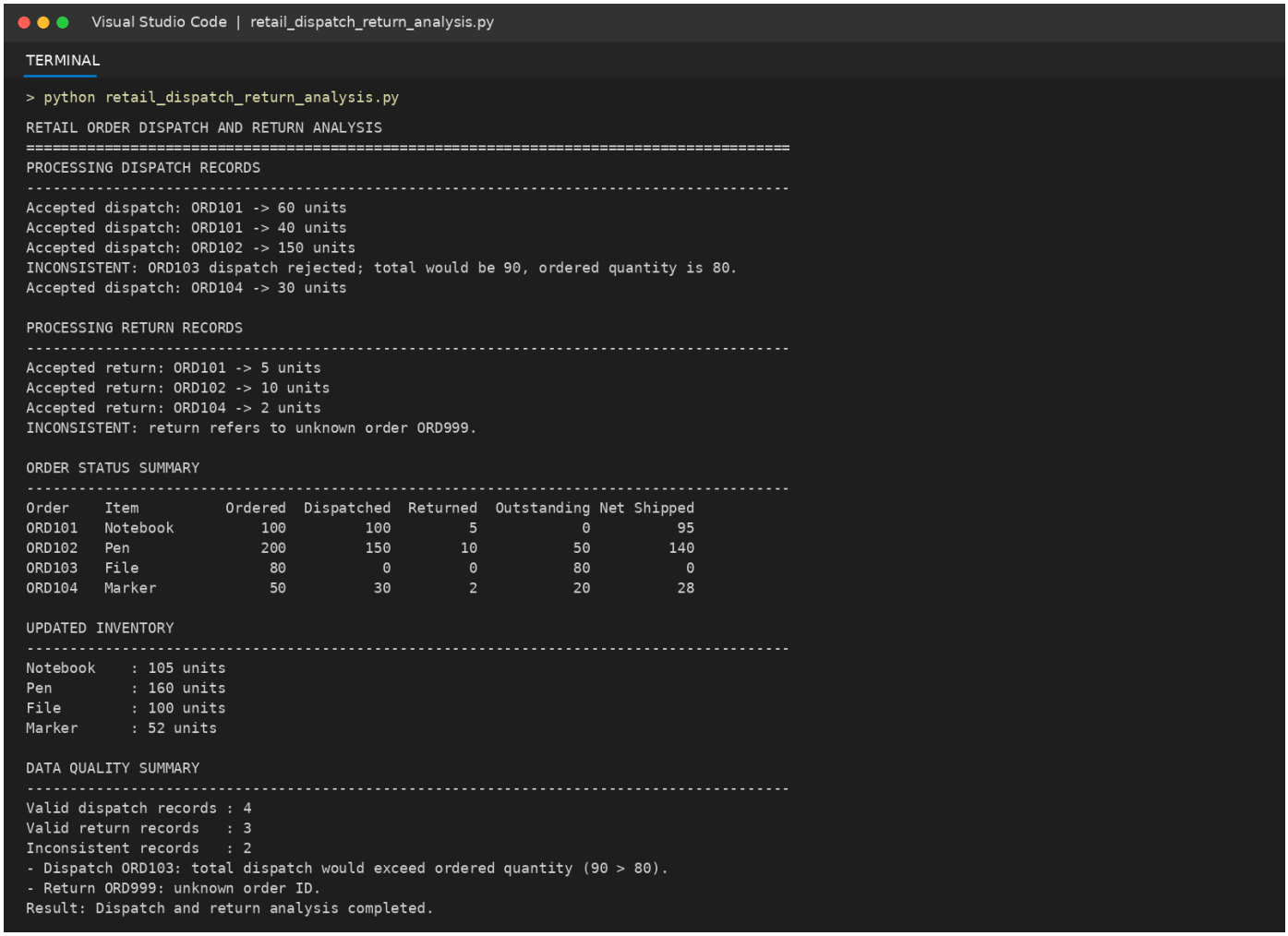
print("\nORDER STATUS SUMMARY")
print("-" * 88)
print(f"{'Order':<9}{'Item':<12}{'Ordered':>9}{'Dispatched':>12}"
      f"{'Returned':>10}{'Outstanding':>13}{'Net Shipped':>12}")
for order_id, data in orders.items():
    dispatched = dispatch_totals[order_id]
    returned = return_totals[order_id]
    outstanding = data["ordered"] - dispatched
    net_shipped = dispatched - returned
    print(f"{order_id:<9}{data['item']:<12}{data['ordered']:>9}"
          f"{dispatched:>12}{returned:>10}{outstanding:>13}{net_shipped:>12}")

print("\nUPDATED INVENTORY")
print("-" * 88)
for item, qty in inventory.items():
    print(f"{item:<12}: {qty} units")

print("\nDATA QUALITY SUMMARY")
print("-" * 88)
print(f"Valid dispatch records : {len(valid_dispatches)}")
print(f"Valid return records   : {len(valid_returns)}")
print(f"Inconsistent records   : {len(issues)}")
for issue in issues:
    print(f"- {issue}")
print("Result: Dispatch and return analysis completed.")
```

7. Sample Execution Output Screenshot

This terminal-style screenshot shows the sample program output. Run the included Python file in Visual Studio Code to reproduce it.



8. Test Cases

1. ORD101: 60 + 40 dispatched against 100 ordered -> accepted.
2. ORD103: dispatch 90 against 80 ordered -> rejected as inconsistent.
3. ORD999 return: unknown order ID -> rejected.
4. ORD101 return 5 against 100 dispatched -> accepted.
5. Outstanding = ordered quantity - accepted dispatched quantity.
6. Inventory = starting stock - accepted dispatches + accepted returns.

9. Conclusion

The program validates dispatch and return records, calculates outstanding and net-shipped quantities, updates inventory consistently, and highlights invalid or inconsistent records for review.